MRL-SPL Team Research Report 2017

Mohammad Ali Sharpasand*, Novin Shahroudi, Mostafa Hassanpour Divshali, Ali Sirouszia, Sina Moqadam Mehr, Masoud Khairi Atani, Mohammadreza Hassanzadeh, Ali Piry, Erfan KouzehGaran, Farzad Fathali Bigelow, Sara SafarAbadi, Nastaran Zareie

Mechatronics Research Laboratories, Qazvin Islamic Azad University, Qazvin, Iran
http://mrl-spl.ir
*a.sharpasand@mrl-spl.ir

Contents

1 Motion Control 3
- 1.1 Automatic Joint and Camera Calibration . . . 3
  - 1.1.1 Modeling and Formulation . . . 5
  - 1.1.2 Observability Analysis . . . 6
  - 1.1.3 Least Square Solution . . . 8
  - 1.1.4 Experimental Results . . . 8
- 1.2 Stability Controller . . . 8

2 Behavior Control 12

3 Future Works 15
- 3.1 Automatic Joint and Camera Calibration . . . 16
- 3.2 Walking Engine . . . 16
- 3.3 Self-Localization . . . 16
- 3.4 Multi-agent Coordination . . . 17
- 3.5 Upcoming Publications and Releases . . . 17

References 20

1

Motion Control

1.1 Automatic Joint and Camera Calibration

NAO Robots need to be re-calibrated after every one or two SPL matches in order to maintain a level of repeatability for camera projection and walk. Usually this calibration is broken into two parts: calibration of camera extrinsic parameters and calibration of joint offsets. Camera calibration is common among most of the teams and a variety of methods are

applied for this purpose, including manual and automatic ones. However, doing joint calibration is not very popular due to the hurdles in the way of accurately measuring offsets. Officially, there is a calibration method done by the manufacturer using a precisely built template in which all of the robot joints are fixed at known positions. Unfortunately, this approach is not very convenient since it should be done only by the support staff and it is not possible to perform such a procedure after every game. There is a manual joint calibration procedure presented by B-Human in their code release but this procedure needs high skills and plenty of time. It can also result in overheating the robot if done right before a game. B-Human has also published a research on automatic calibration of joints offsets and camera extrinsic parameters[3]. In this work, markers are attached to the feet of the robot and they are observed by robot camera in some configurations, then an optimization problem is shaped to find calibration parameters. But the resulting offsets could not completely obsolete manual calibration. Authors of the paper has noted that one of the sources of uncertainty can be joint backlash. Another research on calibrating NAO is done by Maier et al.[5]. In this paper, there are markers attached to the feet and hands of the NAO robot and after collecting a big pool of samples by the camera, a selection algorithm finds the samples that maximize observability of the system. Then offsets are extracted by optimization. Our research is focused on a similar approach, the robot stands with its feet fixed in a template attached to a marker. To simplify the problem, the marker and the template are built to be accurate and precise enough to remove the need to additional parameters for marker displacement. To avoid effect of backlash on the performance of the calibration, the samples are taken near to robot working configuration. An observability analysis helped us to avoid taking many samples by knowing minimal set of samples to shape a fully observable prob-

lem. The offsets are then extracted using the method of Levenberg–Marquardt.

1.1.1 Modeling and Formulation

The most important parameters to estimate are deviation of measured joint angles from the real ones and camera extrinsic parameters all of which can be considered constants. Therefore joint angles are modeled as follows:

$$\vartheta_i = q_i + \delta q_i \tag{1.1}$$

where ϑ_i is the real joint angle, q_i is sensor value and δq_i is the calibration offset for joint i.

For correction of camera extrinsic parameters, Euler rotation representations are used to represent camera disposition. Including above parameters in calculation of camera position, projection of a point in the image vary with changing calibration parameters. Since the robot feet are fixed in the template and we know accurate position of 9 points on the marker relative to robot feet, we can shape a kinematic loop:

$$\vec{r}_{p/f} = \vec{r}_{p/c} + \vec{r}_{c/f} \tag{1.2}$$

where $\vec{r}_{p/f}$ is the position of points on the marker relative to a foot, $\vec{r}_{p/c}$ is the position of camera relative to the foot, and $\vec{r}_{c/f}$ is the position of point on the marker relative to camera as detected in the image and projected on the ground. The above equation holds true only if the calibration parameters are correct, therefore we have to minimize the following residuals:

$$\vec{res} = \vec{r}_{p/c} + \vec{r}_{c/f} - \vec{r}_{p/f} \tag{1.3}$$

1.1.2 Observability Analysis

The critical point of calibration is to ensure the parameters are fully observable; It is important to find a solution to the optimization that is optimal in all configurations and not only in the sampling configuration. In calibrating a whole humanoid there are 20 parameters and each configuration for sampling can at most provide us with 6 independent equations. Therefore, it is impossible to find a single configuration that satisfies observability of the parameters. Most of the studies in this area are focused on selecting the samples that maximize observability from a big pool of samples which is hard to execute on a humanoid robot.[11,7,6,2] We have focused on the conditions of shaping an observable system and the criteria that we should consider for selecting sampling configurations.

Manual camera calibration hinted us rotating camera will show difference between calibration error of camera and inertia sensor. It helped us start from a guess: rotating every 6 degrees of freedom, results in different effects of calibration errors in final error and construct independent equations which increase system observability. More precisely, rotating body torso in two independent directions, completes rank of the resulting problem and ensures there is only one answer to the equation 1.2. Starting from this guess, we have done a careful proof that the problem is fully observable if samples are taken with different torso rotations. The proof is out of scope of this report and will be published separately. Figure 1.1 shows our sampling configurations.

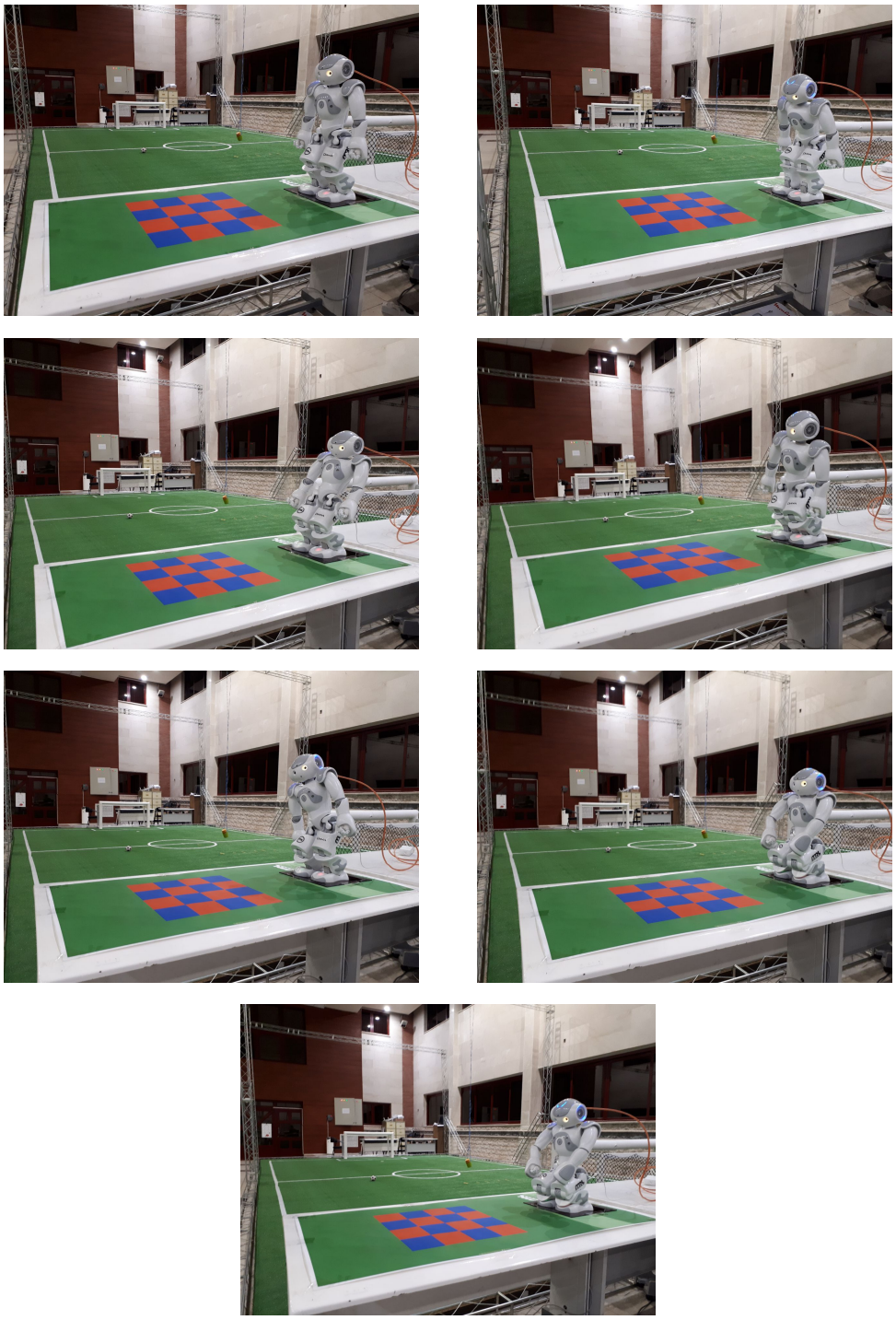

Figure 1.1: Different torso rotations complete the rank of resulting equation systems and enables us to fully observe calibration parameters.

1.1.3 Least Square Solution

Up to this point, the residuals are calculated and we have made sure there is only one minimal solution to it. Due to high non-linearity of the problem, we have chosen the method of Levenberg–Marquardt to solve for least square solution. We have used *Ceres Solver*[1], a non-linear least square solver library which includes the Levenberg–Marquardt with three different types of differentiation, numeric differentiation, analytical Jacobian, and automatic derivatives which calculates exact derivatives using the method of Ratkowsky[8]

1.1.4 Experimental Results

We have implemented this calibration method using numerical differentiation and done experiments in simulation and on real robots. The method converges consistently to correct results but not with sufficient precision. The error stems from numerical round-off errors in numeric differentiation and the method needs more accurate derivatives to proceed further to the solution. Therefore, we are currently working on using Ratkowsky exact derivatives and analytical Jacobian. Implementation and experiments on these approaches has not finished yet. The whole code is developed open-source and is available on GitHub[10] as a separate project which is interfaced to B-Human framework. (interface utilities is also available in the repository.)

1.2 Stability Controller

Last year, we switched from B-Human walking engine to the UNSW Sydney's. The original balance controller in this walking engine was a P-controller with an input of rotational

velocity measured by gyro sensor smoothed using an alpha filter and an output of position displacement. Since the input of the controller was of the type of derivative of the output, we can see the controller as a D-controller as well. Looking from a D-controller perspective, there is no specific control to keep the robot upright. Therefore, we have changed it to a PD to meet stability criteria of walking engine. We tried Tyreus Luyben method to tune the parameters but could not get an appropriate result. So, we established manual method and tuned P and D parameters. In this regard, we could meet 350 mm/step speed instead of 300. An excerpt of out code is presented to show the exact change. It can also be applied as a patch into UNSW walking engine:

```
float thetaY = sensors.sensors[InertialSensor_AngleY];
const float thetaDotY = sensors.sensors[InertialSensor_GyrY];

const float thetaYRef = 0;
const float thetaDotYRef = 0;
float errorThetaY = thetaYRef - thetaY;
const float errorThetaDotY = thetaDotYRef - thetaDotY;

thetaYHistory.push_front(crop(errorThetaY, balanceParameter.MinThetaY,
    balanceParameter.MaxThetaY));

const float theBound = 0.09;
balanceAdjustment = balanceParameter.YP * (errorThetaY - 0.93 *
    crop(errorThetaY, -theBound, theBound)) + balanceParameter.YI *
    thetaYHistory.sum() * 0.01 + balanceParameter.YD * errorThetaDotY;
```

```
    float thetaX = sensors.sensors[InertialSensor_AngleX];
  const float thetaDotX = sensors.sensors[InertialSensor_GyrX];


  const float thetaXRef = 0;
  const float thetaDotXRef = 0;
  float errorThetaX = thetaXRef - thetaX;
  const float errorThetaDotX = thetaDotXRef - thetaDotX;
 thetaXHistory.push_front(errorThetaX);
   const float theOtherBound = 0.;
   coronalBalanceAdjustment = balanceParameter.XP * (errorThetaX - 0.6 *
       crop(errorThetaX, -theBound, theBound)) + balanceParameter.XI *
       thetaXHistory.sum() * 0.01 + balanceParameter.XD * (errorThetaDotX +
       crop(errorThetaDotX, -theOtherBound, theOtherBound) * 2);
```

Four experiments are designed to compare the performance of the new and the original controllers under the same disturbances: walking on an inclined plane, maintaining stability during collision to an obstacle in high speed walking, recovering from an impulse during walking in place, and during kick. There was no significant change in first two experiments. In impulse tests, we applied several disturbances to the robot with a pendulum from different distances during walk and kick. Our controller recovered slightly faster than UNSW's in impulse tests. You can see robot torso angle during walk impulse test in Figure 1.2 and during kick impulse test in Figure 1.3

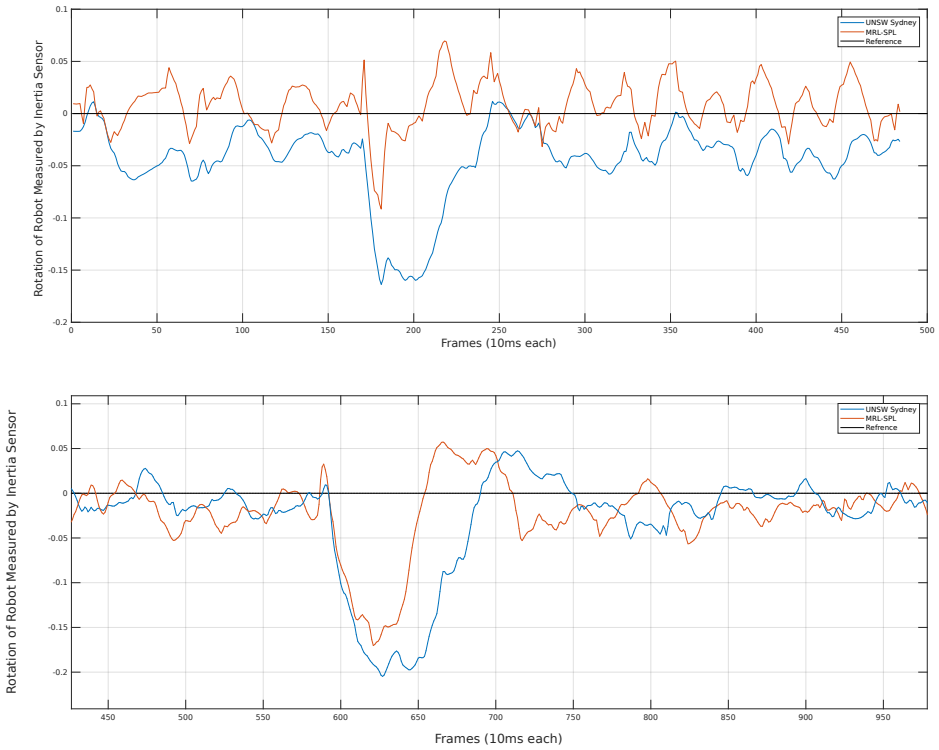

Figure 1.2: UNSW original (blue) and updated (red) stability controller recovering from two impulse of different magnitude applied from front to the robot while it was stepping in place.

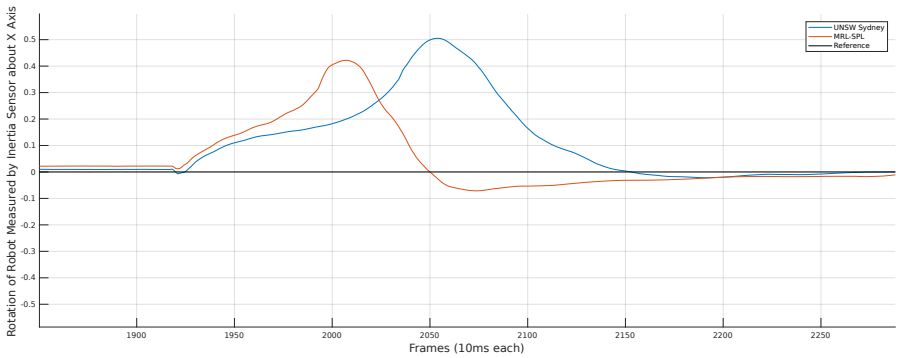

Figure 1.3: UNSW original (blue) and updated (red) stability controller recovering from an impulse applied from side to the robot during a kick.

2

Behavior Control

Our behavior control contains two parts, low-level and high-level. Low-level part consists of fundamentals of robot's behavior such as walking to the ball or to a specific position, and dribbling opponent. High-level part includes task assignment and some other sections. Task assignment module is being employed since 2015 competitions. From then, we have developed many features and improved the module. This module is mainly inspired by the

work of MacAlpine, et al., "Positioning to Win"[4] at Simulation3D league.

Task assignment is a module which coordinates the team and assigns a task to each robot. Each task can be either post or role. Post is the long term task of an agent, role is a short term task. Post is analogous to positions on the field such as Goal Keeper, Defender, Halfback, Forward, etc. that are defined on the formation as static points. On the other hand, a role corresponds to a position relative to the ball such as Leader and Supporter roles. Role positions are not predefined in the formation, rather calculate in real-time.

Last year, we have made some new changes in Task assignment. One of which, was a new feature called leader2 task. During the game, there are some circumstances that the leader may fall behind the opponent. To that end, another agent that is closer to the ball will become second leader together with the first leader until the one which is closer be in front of the opponent not behind of it.

We also try to get benefit of the kick off as an opportunity to score a goal. When kick off is with us, the leader player passes the ball to the "palang" robot which goes to its position in opponent half, receives the pass and kicks toward the goal. "Palang", which means Leopard, is a symbol of being swift. It is a post position in the formation that is placed close to the side lines in the game ready state (Figure 2.1) and when game starts it is placed in the opponent's half to increase its chances to receive a pass (Figure 2.2). The passing mechanism is part of the pass planner module and is out of the scope of this report. However, it worth to mention that the pass planner works in a passive fashion so that every agent finds best passing option to other agents with respect to different circumstances and stages of the game so that no communication is required between these two modules at the formation or any other level.

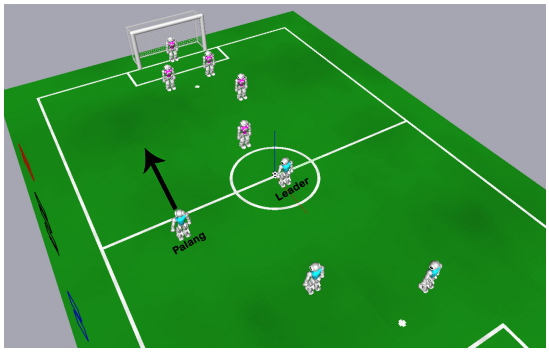

Figure 2.1: Task assignment Palang scenario in set game state

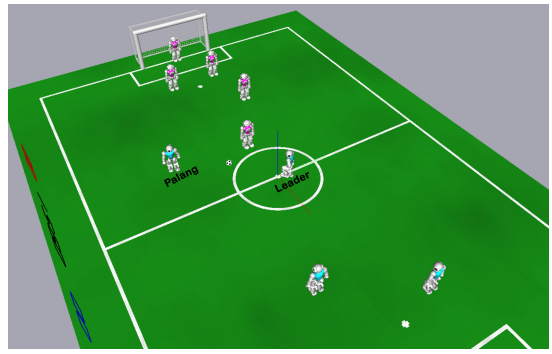

Figure 2.2: Task assignment Palang scenario in playing game state

Task	Description	Position
Leader	Closest agent to the ball	Low level motion planning
Leader2	Second closest agent to the ball	Low level motion planning
Supporter	Second closest agent to the ball	Low level motion planning
Defender	A post on the defense line	Low level motion planning
Palang	For the kickOffUs scenario	static point on formation
Goal Keeper	Always assigned to player 1	Low level motion planning

Table 2.1: List of currently employed tasks and their description

3

Future Works

To follow up our previous researches and add up some more value to our team work activities, we are planning to work on Automatic Joint and Camera Calibration, Walking Engine, Self-Localization, and Multi-agent Coordination. Each section is discussed in detail.

3.1 Automatic Joint and Camera Calibration

Our research on automatic calibration continues. It is being constantly updated on GitHub. The first step is to try solving the least square with a more precise derivative, rather analytical Jacobian. Then it has to be enhanced a little to be more user friendly, and finally an extensive paper on the details of the approach, proof of observability, and results will be published as well as a users guide.

3.2 Walking Engine

We are doing an ongoing research on walking engines. After porting UNSW's walking engine to our code, we are also working on Nao Devil's engine to compare them accurately and probably doing contributions to them. Our plan includes implementing several different control strategies into these walking engines to achieve more stability.

On the other hand, our work on a portable biped walking engine is continued in which, we try to implement a Zero Moment Point (ZMP) preview controlled walking engine that can easily be ported into different codes and applied to different biped robots which have physical characteristics required by a ZMP-walker. The code for this engine will also be developed open-source on our team GitHub page and will be announced as soon as it reaches the first usable version.

3.3 Self-Localization

Having free-kick rules, there is a need for more coordination between robots which means a need to more accurate world model. Current self-localization method, the one released

in B-Human code release 2016[9], is not robust enough especially in recovering from kidnap scenarios to shape accurate enough world modeling. Therefore, we plan to use our own self-localization module using Adaptive Monte Carlo Localization to overcome troubles in mis-localization. On the other hand, current version is very dependent on correct and precise calibration of camera extrinsic parameters, which can be reduced by using a better strategy for overcoming the challenge of symmetric field and modeling distance error caused by mis-calibration inside the measurement model. A similar strategy is also taken in the current localization module (B-Human's) but it is currently inevitable not to weigh far landmarks less than what it is weighed.

3.4 Multi-agent Coordination

The formation formulation can be improved to make it more usable for dynamic scenarios. At the same time, other multi-agent coordination algorithms can be studied and evaluated to cover shortcomings of the currently employed approach. A good approach would be the one that does not depend on the position of agents directly. Among those, deep learning or distributed auction algorithm might be the good candidates. Furthermore, quantitative assessment of the performance especially regarding the synchronization rate is required for validation of the method.

3.5 Upcoming Publications and Releases

In order to do our part in developing Standard Platform League, we are working hard to separate different parts of our code as stand-alone modules and publish them. Upcoming publications and code-releases of our team will be:

- Automatic Joint and Camera Calibration
- Portable Walking Engine
- Portable Self-Locator
- Visual Attention Module (Overt and Covert)
- Motion Planning

References

[1] Agarwal, S., Mierle, K., & Others (2018). Ceres solver. http://ceres-solver.org.

[2] Carrillo, H., Birbach, O., Täubig, H., Bäuml, B., Frese, U., & Castellanos, J. A. (2013). On task-oriented criteria for configurations selection in robot calibration. In *Robotics and Automation (ICRA), 2013 IEEE International Conference on* (pp. 3653–3659).: IEEE.

[3] Kastner, T., Röfer, T., & Laue, T. (2014). Automatic robot calibration for the nao. In *Robot Soccer World Cup* (pp. 233–244).: Springer.

[4] MacAlpine, P., Barrera, F., & Stone, P. (2013). Positioning to win: A dynamic role assignment and formation positioning system. In *RoboCup 2012: Robot Soccer World Cup XVI* (pp. 190–201). Springer.

[5] Maier, D., Wrobel, S., & Bennewitz, M. (2015). Whole-body self-calibration via graph-optimization and automatic configuration selection. In *2015 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 5662–5668).: IEEE.

[6] Nahvi, A. & Hollerbach, J. M. (1996). The noise amplification index for optimal pose selection in robot calibration. In *Robotics and Automation, 1996. Proceedings., 1996 IEEE International Conference on*, volume 1 (pp. 647–654).: IEEE.

[7] Nahvi, A., Hollerbach, J. M., & Hayward, V. (1994). Calibration of a parallel robot using multiple kinematic closed loops. In *Robotics and Automation, 1994. Proceedings., 1994 IEEE International Conference on* (pp. 407–412).: IEEE.

[8] Ratkowsky, D. (1983). Nonlinear regression modeling. http://www.itl.nist.gov/div898/strd/nls/data/ratkowsky3.shtml.

[9] Röfer, T., Laue, T., Kuball, J., Lübken, A., Maaß, F., Müller, J., Post, L., Richter-Klug, J., Schulz, P., Stolpmann, A., Stöwing, A., & Thielke, F. (2016). B-Human team report and code release 2016. Only available online: http://www.b-human.de/downloads/publications/2016/coderelease2016.pdf.

[10] Sharpasand, M. A., Mehr, A. M., Bigelow, F. F., & Harandi, M. A. Z. (2018). Open-source library for biped walking and dynamics. https://github.com/mrlspl/biped-library.

[11] Sun, Y. & Hollerbach, J. M. (2008). Active robot calibration algorithm. In *Robotics and Automation, 2008. ICRA 2008. IEEE International Conference on* (pp. 1276–1281).: IEEE.